Movie Recommendation System

via Tau Prolog

Team members

921230001	ابراهيم احمد حمدي
921230035	جون جمال شحاته

Technical Report: AI-Powered Movie Recommendation System

1. Project Overview

The objective of this project is to design and implement an AI-based Expert System that acts as a Movie Recommender. The core logic and knowledge base are built using **Prolog** (Programming in Logic), which excels at pattern matching, declarative rule-based logic, and recursive data processing. To make the system accessible and user-friendly, the Prolog backend is seamlessly integrated into a modern web interface (HTML/CSS/JavaScript) using the **Tau Prolog** library, allowing the entire application to run directly within a web browser without requiring a dedicated backend server.

2. System Architecture

The system adopts a 100% client-side, serverless architecture.

The architecture consists of three main layers:

1. **The Knowledge Base (Prolog):** Contains the dataset (facts) and the filtering logic (rules).
2. **The Interface (HTML/CSS):** Provides the visual elements, quick-filter buttons, and search input fields for the user.
3. **The Middleware (JavaScript & Tau Prolog):** Acts as the bridge. JavaScript captures user inputs, constructs Prolog queries, passes them to the Tau Prolog interpreter running in the browser, and then dynamically renders the answers back to the HTML DOM.

3. Implementation Details: The Prolog Backend

The Prolog code is the brain of the project, divided into three fundamental programming concepts:

3.1. Facts (Data Representation)

The system relies on a rich dataset represented by the `film/10` predicate. Each fact holds ten specific arguments defining a movie's attributes: `film(Title, Director, Year, MainGenre, SubGenre, Duration, Country, LeadActor, Rating, Votes).` Additionally, unary facts like `actress('Nicole Kidman').` were defined to classify specific actors, allowing the system to deduce further information (e.g., distinguishing male actors from female actresses using the `\+` negation operator).

3.2. Rules (Logical Filtering)

Rules were constructed to establish inferential relationships and filter the dataset based on specific conditions. Key implementations include:

- **Direct Attribute Filtering:** Rules like `genre/2` and `rating/2` isolate specific arguments from the complex `film/10` fact.
- **Mathematical Conditions:** Rules like `short_movie/1` (Duration < 100) and `released_after/2` (Year > YearLimit) utilize comparison operators to filter movies.
- **Compound Rules:** The `masterpiece/1` rule demonstrates code reusability and logical AND operations by combining the `high_rating/1` and `popular_movie/1` rules.

3.3. Advanced Logic (Lists and Recursion)

To demonstrate advanced logic programming paradigms, the system utilizes Prolog's list processing capabilities:

- **List Building (`findall`):** The `director_movies/2` and `female_lead_movies/1` rules use the `findall/3` built-in predicate to gather all movies satisfying a specific condition into a single, unified list.
- **Recursion:** The `total_duration/2` and `count_movies/2` rules utilize recursive step logic (splitting lists into Head and Tail) and base cases (empty lists) to mathematically aggregate data from generated lists.

4. Implementation Details: The Web Frontend

To move away from the traditional command-line console, a modern graphical user interface was developed.

4.1. UI Design (HTML & CSS)

The interface features a dark-themed, cinematic design layout. It is divided into two primary sections:

- **Quick Filters Panel:** Contains pre-configured buttons triggering standalone queries (e.g., "Masterpieces", "Epic Movies").
- **Custom Search Panel:** Contains interactive forms, including dropdown menus for genres and input fields for numerical data (Year, Rating) and text data (Director).

4.2. Tau Prolog Integration (JavaScript)

The integration is executed via a script element loading `tau-prolog.js`. The operational flow is programmed as follows:

1. **Session Creation:** `var session = pl.create();` initializes the Prolog engine environment.
2. **Consultation:** The entire Prolog code (facts and rules) is stored in a JavaScript string variable (`prologCode`) and loaded into the engine using `session.consult(prologCode);`.
3. **Dynamic Query Execution:** A unified JavaScript function, `runQuery(queryStr, titleText)`, handles all interactions. When a user clicks a search button, this function formulates the exact Prolog query (e.g., `released_after(Movie, 2010).`) and executes it via `session.query()`.

4. **Answer Handling:** The `session.answers()` callback mechanism asynchronously traverses the resolution tree, extracting the bound variables (e.g., the Movie Title), sanitizing the output string (removing quotes), and dynamically appending the results to the HTML document.

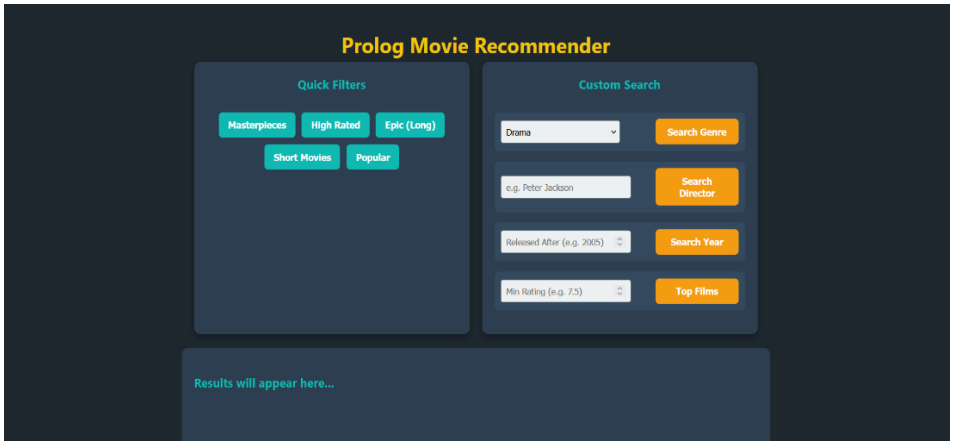
5. Conclusion

This project successfully demonstrates the practical application of Artificial Intelligence concepts through Logic Programming. By writing a robust Knowledge Base in Prolog and creatively embedding it into a web environment using Tau Prolog, the project bridges the gap between traditional AI logic engines and modern, accessible user interfaces. The resulting system is a fully functional, highly responsive, and scalable movie recommendation engine.

6. Appendix A: System Screenshots

6.1. Backend Logic & Console Execution

```
28 ?- movie(Title).
Title = 'The Green Mile' ;
Title = 'Bang Bang! You\'re Dead!' ;
Title = 'Dogville' ;
Title = 'Detachment' ;
Title = 'Schindler\'s List' ;
Title = 'Requiem for a Dream ' .

29 ?- genre('The Matrix', Genre).
Genre = adventure ;
Genre = scifi.

30 ?- rating('Shrek', Score).
Score = 7.8.

31 ?- likes(Person, Genre).
Person = 'Ahmed',
Genre = scifi ;
Person = 'Salma',
Genre = drama ;
Person = 'Kareem',
Genre = thriller.

32 ?- actress('Natalie Portman').
true.

33 ?- high_rating(Movie).
Movie = 'The Green Mile' ;
Movie = 'Schindler\'s List' ;
Movie = 'Gladiator' ;
Movie = 'The Pianist' ;
Movie = 'Forrest Gump' .

34 ?- short_movie(Movie).
Movie = 'Bang Bang! You\'re Dead!' ;
Movie = 'Shrek' ;
Movie = 'Ice Age' .
```

```
35 ?- released_after(Movie, 2005).
Movie = 'Detachment' ;
Movie = 'Biutiful' ;
Movie = 'Black Swan' ;
Movie = 'The Hangover' .

36 ?- classic_film(Movie, 1990).
Movie = 'Sexmission' ;
Movie = 'Home Alone' ;
Movie = 'Teddy Bear' .

37 ?- long_movie(Movie, 140).
Movie = 'The Green Mile' ;
Movie = 'Dogville' ;
Movie = 'Schindler\'s List' .

38 ?- recommend('Salma', Movie).
Movie = 'The Green Mile' ;
Movie = 'Bang Bang! You\'re Dead!' ;
Movie = 'Dogville' .

39 ?- recommend('Kareem', Movie).
Movie = 'The Silence of the Lambs' ;
Movie = 'The Da Vinci Code' .

40 ?- movies_in_genre('horror', List).
List = ['The Shining', 'I Am Legend', 'Interview with the Vampire', 'Alien', 'The Ring', '
Saw', 'Final Destination', 'The Ninth Gate', 'The Conjuring'|…].

41 ?- female_lead_movies(List).
List = ['Dogville', 'Black Swan', 'The Devil Wears Prada', 'In Desert and Wilderness', 'Th
e Others', 'Chocolat', 'Step Up', 'Amélie', 'Bridget Jones\'s Diary'|…].

42 ?- epic_movie(Movie), masterpiece(Movie).
Movie = 'The Green Mile' ;
Movie = 'Gladiator' ;
Movie = 'The Lord of the Rings: The Fellowship of the Ring' .
```

6.2. Modern Web Interface (Frontend):